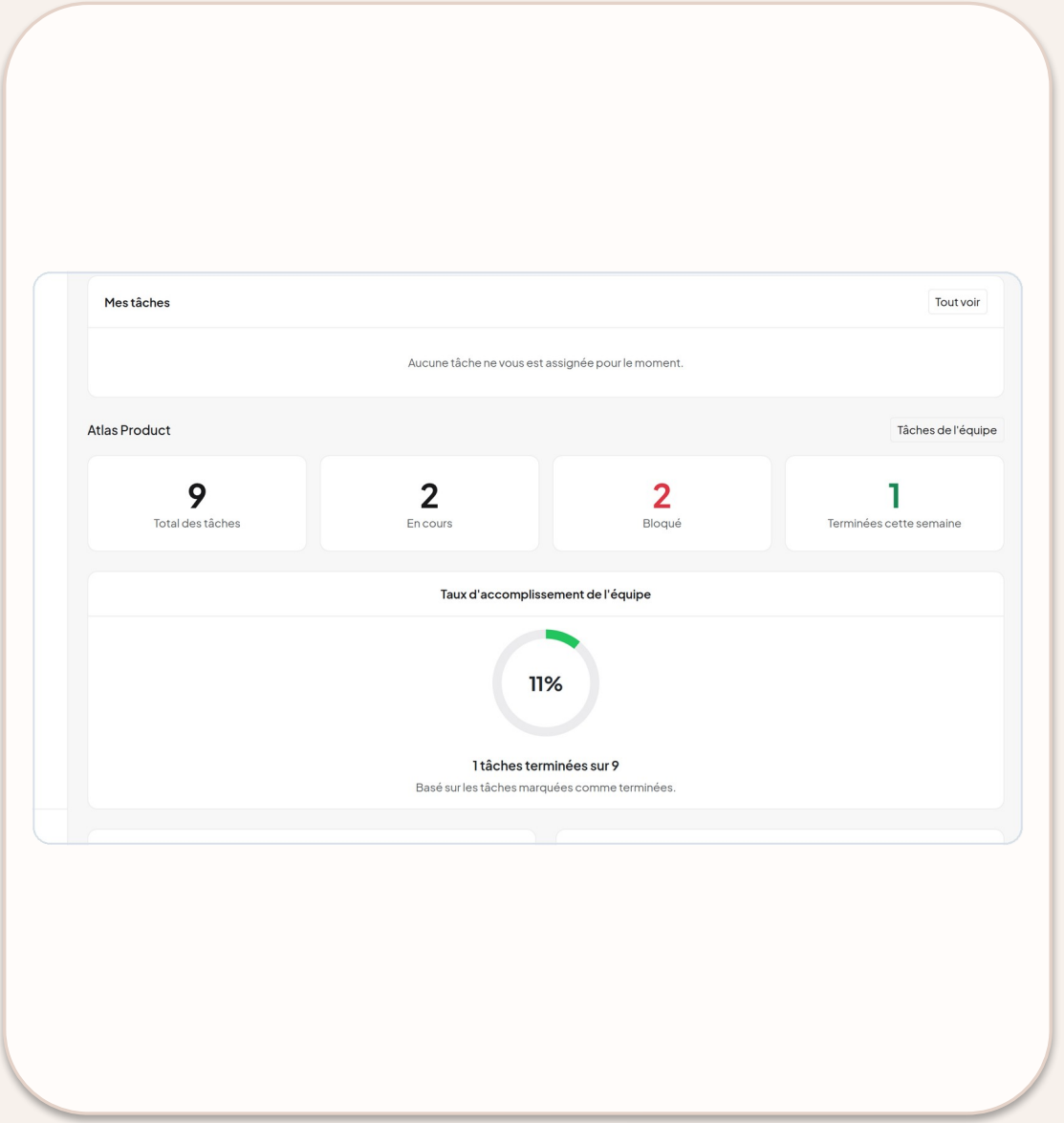
Bachelor concepteur développeur d'applications

Briefly

Le suivi quotidien
d'une équipe, collé à la tâche.

Chiraz Lamraoui
Drif Consult · Yasser El Habib Drif
Anis Benammar · 27 octobre 2026

Oral d'environ 40 minutes



Plan

01 **Contexte** — 10 min
Entreprise, rôle, problème

02 **Analyse** — 8 min
Rôles, cas d'utilisation, activité

03 **Conception** — 8 min
Diagrammes, puis le code

04 **Sécurité et tests** — 8 min
Cinq cas, avec le test

05 **Démonstration et bilan** — 6 min
Application, puis questions

Quarante minutes, cinq parties

Référentiel CDA

Développer

Environnement

Interfaces web et mobile

Composants métier

Scrum

Concevoir

Besoins et cas d'utilisation

Architecture en couches

Base relationnelle

Eloquent

Déployer

Plans de tests

Même contrat web et mobile

Mise en ligne

PHPUnit

RNCP 37873 · trois activités

L'organisme de stage

Entreprise	Drif Consult
Tuteur	Yasser El Habib Drif
Produit du quotidien	Roman.ai, coworker pour Slack
Cadre	CDA, campus Paris, ADG Education

Ce que j'y ai fait

Front-end

Les interfaces du produit.
La cohérence visuelle des écrans.

Parrainage

Relier un utilisateur
à ceux qu'il invite.

Briefly n'est pas une fonction de Roman.ai. C'est le projet du mémoire.

Front-end et parrainage, en général

Le problème observé

1. Le point du matin vit dans la messagerie.
2. Le message n'est pas relié à une tâche.
3. Le blocage disparaît dans le fil.
4. Le chef n'a pas un tableau stable.

Problématique

Comment donner à une organisation fermée un suivi quotidien, attaché aux tâches, sur le web et sur mobile, avec des rôles clairs ?

- **Organisation fermée**
- **Attaché aux tâches**
- **Web et mobile**
- **Rôles clairs**

Ce que le marché ne réunit pas

Messagerie

Slack, Teams

Immédiat.
Rien ne reste sur la tâche.

Tableau léger

Trello

Des cartes.
Pas de note obligatoire.

Forge

Jira, Linear

Des tickets.
Trop lourd pour le matin.

Aucun ne réunit les trois rôles, la note obligatoire, et les deux clients.

Briefly

Organiser

Équipes, projets, tâches

Rendre compte

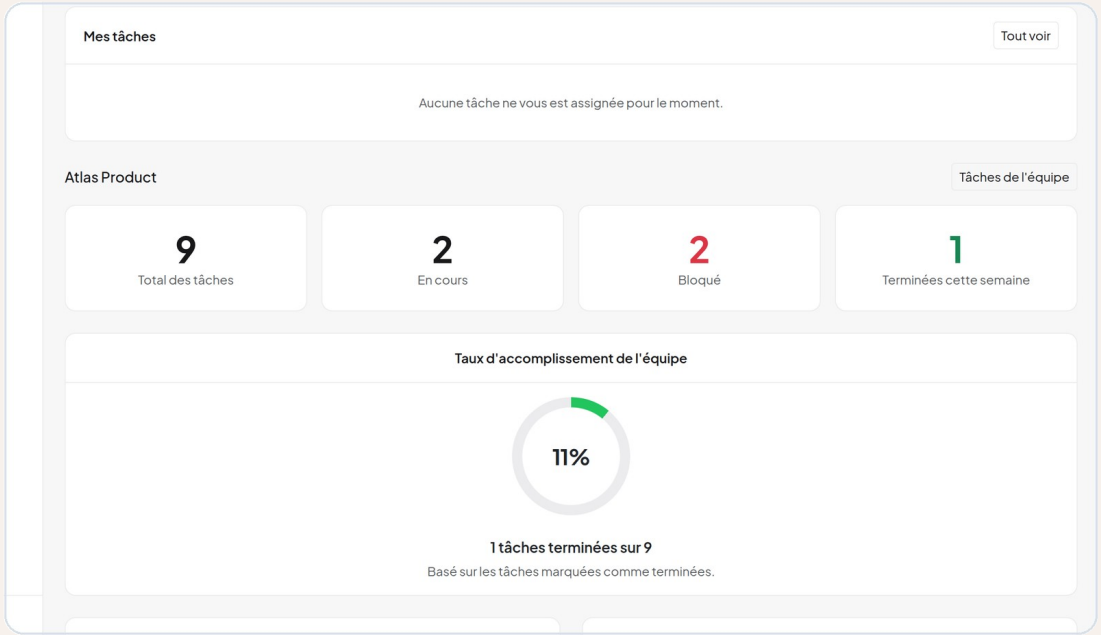
Déjà fait, suite, blocage

Sécuriser

Trois rôles, politiques

Suivre

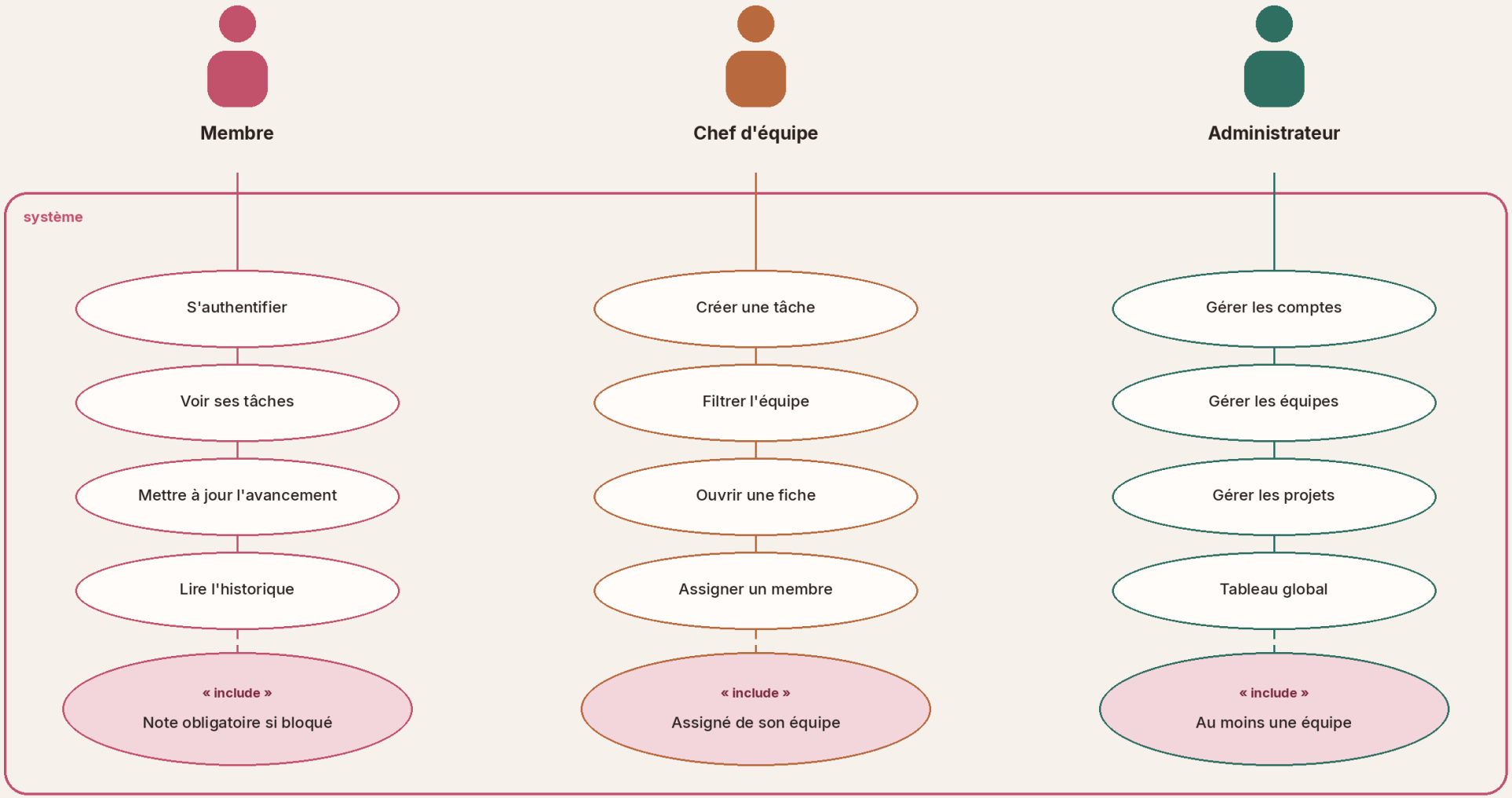
Taux membre, équipe, projet



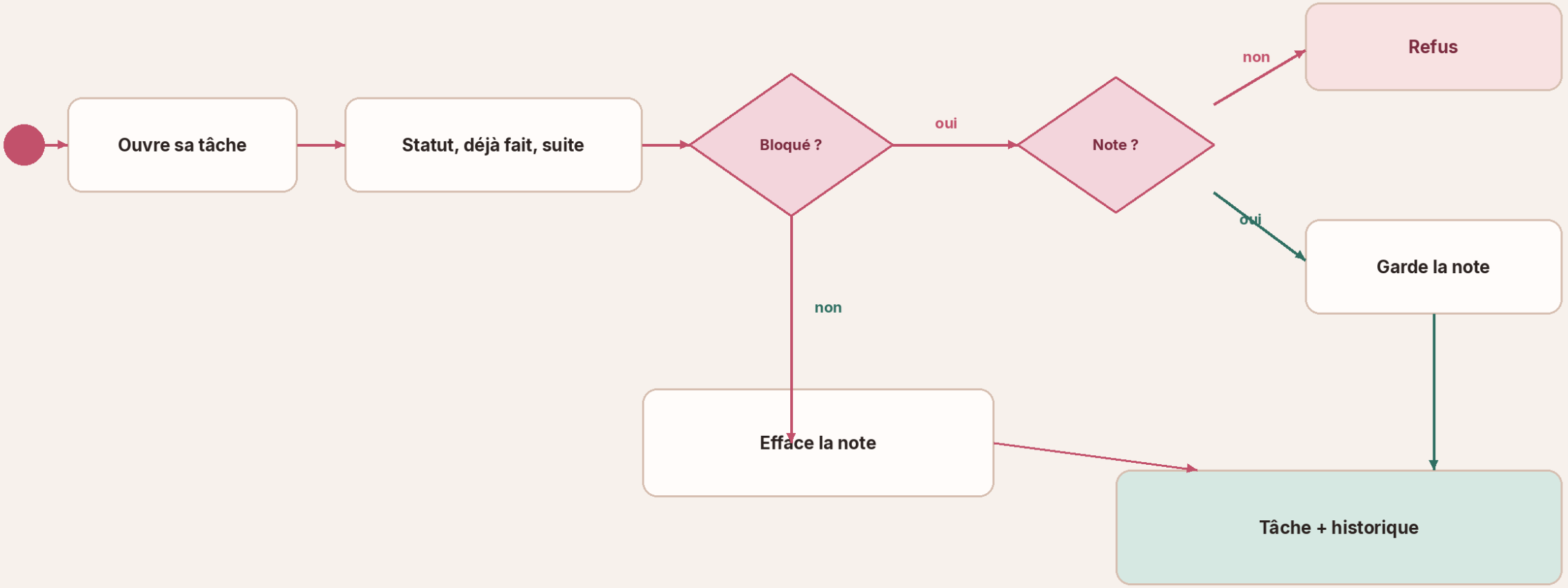
Trois acteurs

Administrateur	Chef d'équipe	Membre
Peut	**Peut**	**Peut**
Comptes, équipes, projets, vue globale.	Tâches de ses équipes, filtre, fiche.	Ses tâches, son point, son historique.
Ne peut pas	**Ne peut pas**	**Ne peut pas**
Pas de tâches personnelles.	Ni les projets, ni les comptes.	Seulement ce qui lui est assigné.

Cas d'utilisation



Activité : mettre à jour l'avancement



Puis : si le statut est terminé, completed_at est posé. Sinon, cette date est effacée.

Scrum

Backlog

La liste priorisée.

Sprint

Un cycle court, un incrément.

User story

Le besoin, dit depuis un rôle.

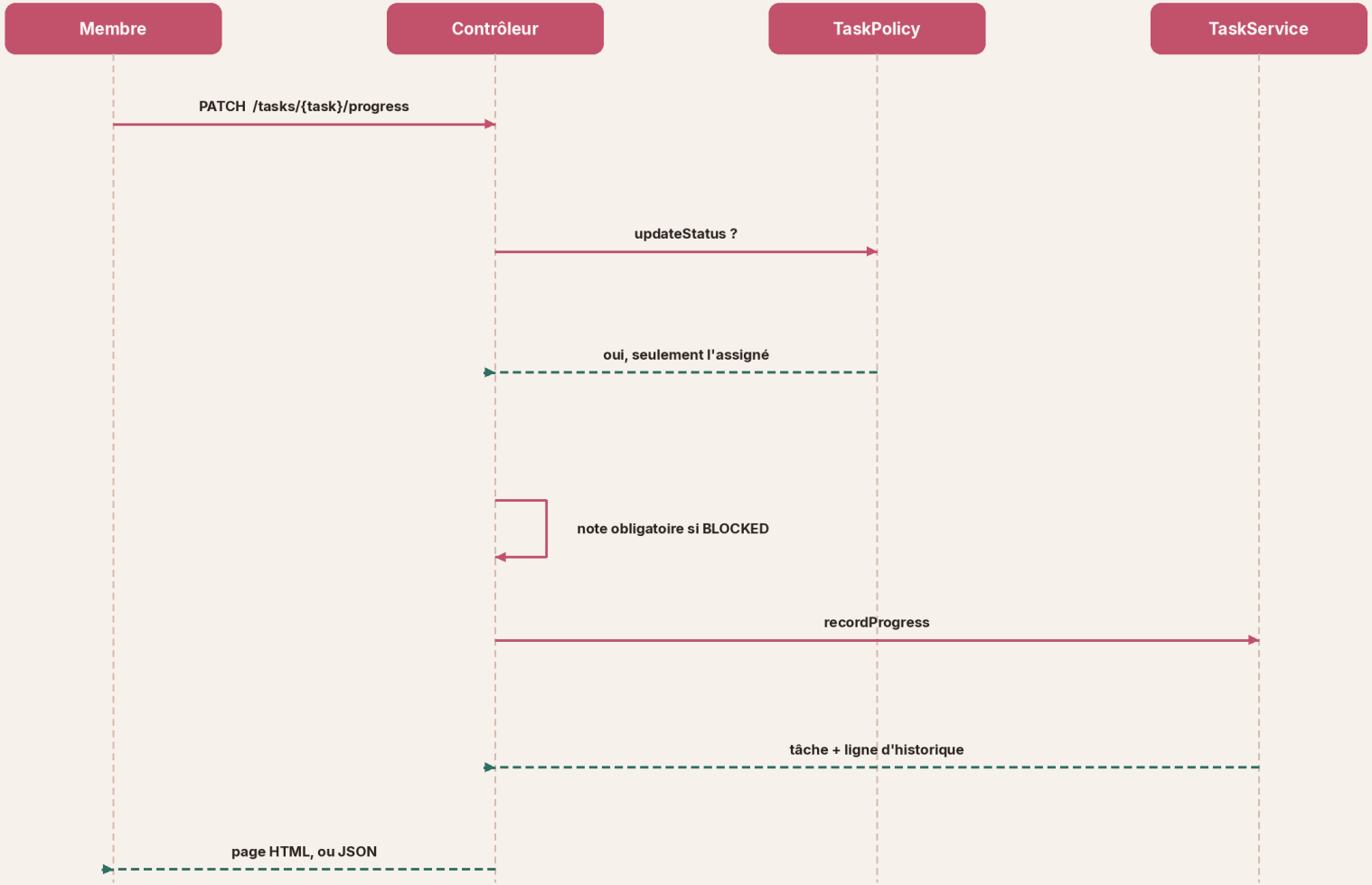
Incrément

Une partie que l'on peut essayer.

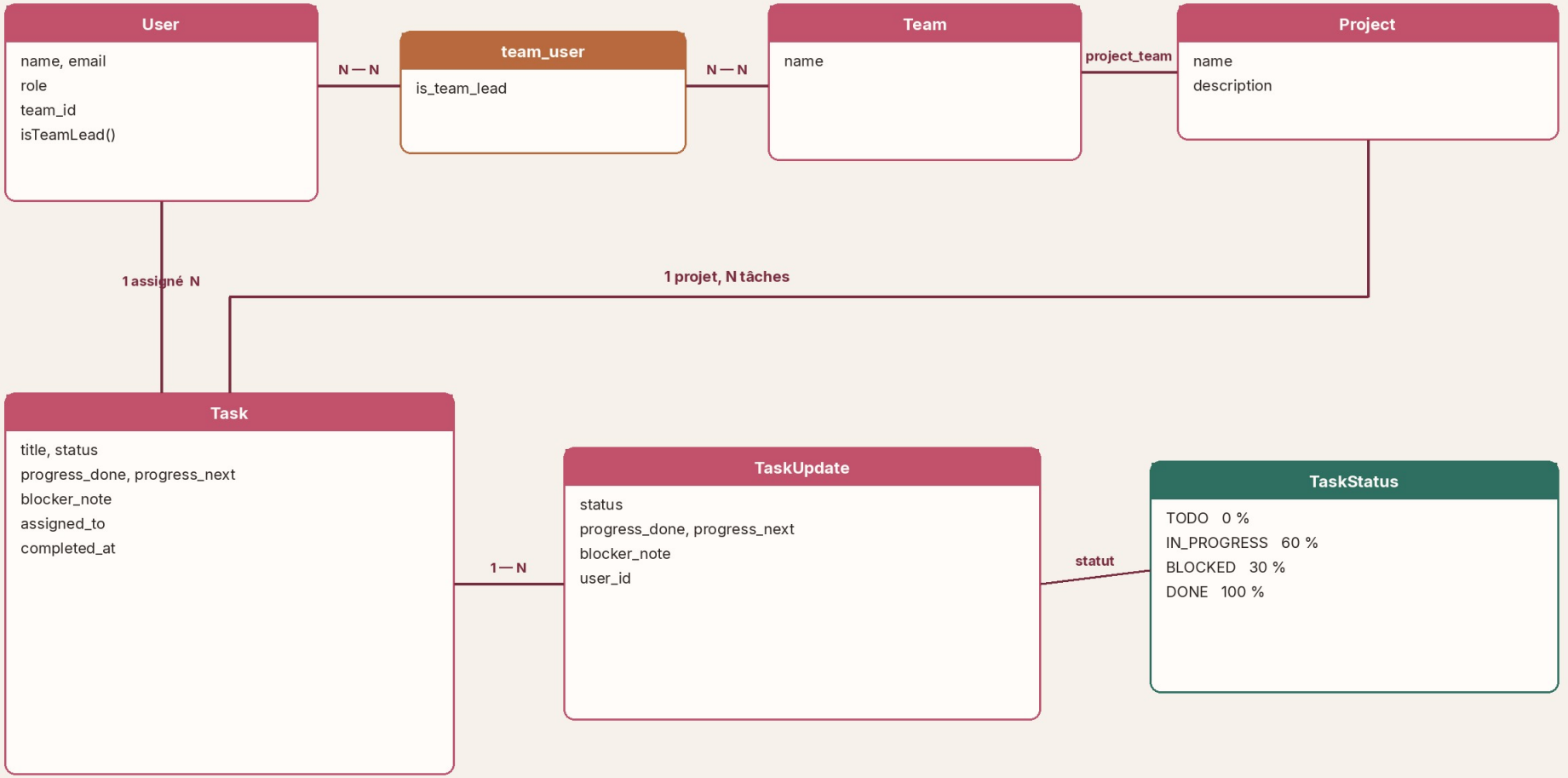
Cinq sprints

S1	2 au 15 février	**Connexion, profil, langue, thème**
S2	16 février au 1er mars	**Espace membre**
S3	2 au 15 mars	**Espace chef d'équipe**
S4	16 au 29 mars	**Administration**
S5	30 mars au 19 avril	**Application Flutter**

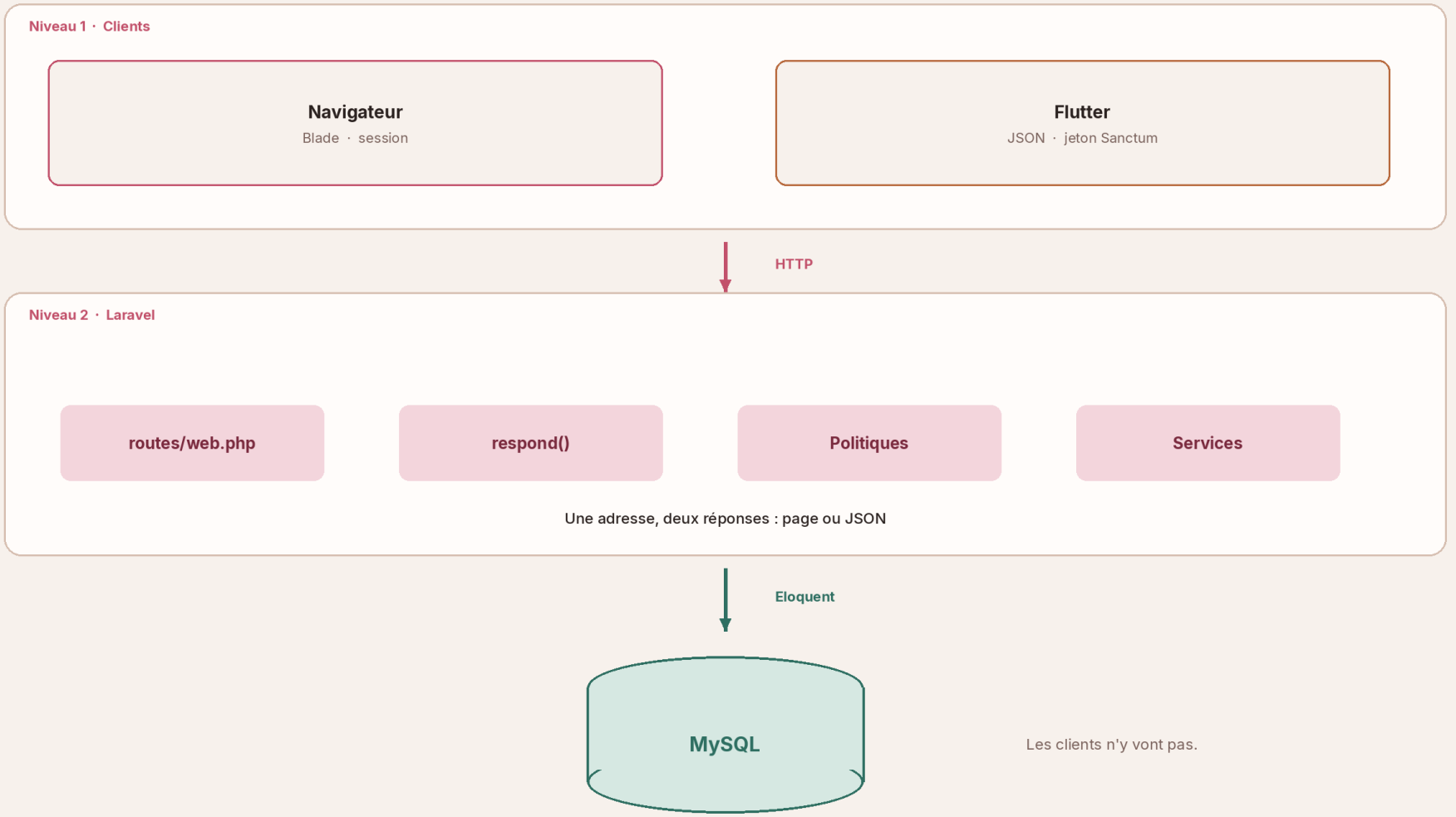
Séquence du point d'avancement



Modèle de données



Architecture 3-tiers



La note devient obligatoire

UpdateTaskProgressRequest.php

```php
public function rules(): array
{
    return [
        'status' => ['required', Rule::enum(TaskStatus::class)],
        'progress_done' => ['nullable', 'string', 'max:5000'],
        'progress_next' => ['nullable', 'string', 'max:5000'],
        'blocker_note' => [
            'nullable', 'string', 'max:5000',
            Rule::requiredIf(fn () =>
                $this->input('status')
                    === TaskStatus::Blocked->value
            ),
        ],
    ];
}
```

Ce que le service enregistre

TaskService.php

```php
$completedAt = match (true) {
    $status === TaskStatus::Done => $task->completed_at ?? now(),
    default => null,
};

$task->update([
    'status' => $status,
    'progress_done' => $data['progress_done'] ?? null,
    'progress_next' => $data['progress_next'] ?? null,
    'blocker_note' => $status === TaskStatus::Blocked
        ? ($data['blocker_note'] ?? null) : null,
    'completed_at' => $completedAt,
]);
```

Deux pourcentages différents

Pourcentage du statut

TaskStatus.php

```php
public function progressPercent(): int
{
    return match ($this) {
        self::Todo => 0,
        self::InProgress => 60,
        self::Blocked => 30,
        self::Done => 100,
    };
}
```

Taux du tableau · terminées / total

TaskService.php · taux du tableau

```php
public function completionSummary(Collection $tasks): array
{
    $total = $tasks->count();
    $done = $tasks->where('status', TaskStatus::Done)->count();

    return [
        'rate' => $total > 0 ? (int) round(($done / $total) * 100) : 0,
    ];
}
```

Le droit, puis la réponse

TaskPolicy::updateStatus

TaskPolicy.php

```php
public function updateStatus(User $user, Task $task): bool
{
    return $task->assigned_to === $user->id;
}
```

Controller::respond

Controller.php

```php
protected function respond(Request $request, $html, $json, int $status = 200)
{
    if (! $request->expectsJson()) {
        return $html;
    }

    return response()->json($json, $status);
}
```

Sécurité

Mots de passe

Hachés, jamais renvoyés.

Session et jeton

Session web, Bearer mobile.

CSRF

Formulaires web. Pas le jeton.

Politique

Le droit se teste sur la ressource.

Validation

Avant toute écriture.

Limite assumée

Pas de blocage des tentatives.
Jetons sans expiration.

Test · sans session

Démarche

Ouvrir le tableau de bord sans être connecté.

Résultat

L'invité est renvoyé vers la connexion. La racine du site aussi.

AuthenticationTest.php

```
1 $this->get(route('dashboard'))
2     ->assertRedirect(route('login'));
```

Test · bloqué sans note

Démarche

Passer une tâche au statut bloqué, sans écrire la note.

Résultat

Le formulaire web refuse. L'appel JSON refuse aussi, en 422.

ProjectTaskTest.php

```php
$this->actingAs($member)
    ->patch(route('tasks.update-progress', $task), [
        'status' => TaskStatus::Blocked->value,
        'progress_done' => 'Started work',
    ])
    ->assertSessionHasErrors('blocker_note');
```

Test · la tâche d'un autre

Démarche

Avec un compte membre, mettre à jour une tâche assignée à quelqu'un d'autre.

ProjectTaskTest.php

```php
$this->actingAs($member)
    ->patch(route('tasks.update-progress', $task), [
        'status' => TaskStatus::Done->value,
    ])
    ->assertForbidden();
```

Résultat

La politique refuse. La réponse est 403.

Test · le chef ne crée pas le projet

Démarche

Avec un compte chef d'équipe, tenter de créer un projet.

Résultat

La création reste à l'administrateur. Le chef peut créer une tâche dans son périmètre.

ProjectTaskTest.php

```php
$this->actingAs($lead)
    ->post(route('admin.projects.store'), [
        'name' => 'Website Redesign',
        'team_ids' => [$team->id],
    ])
    ->assertForbidden();
```

Test · JSON sans jeton

Démarche

Appeler le tableau de bord en JSON, sans le jeton Sanctum.

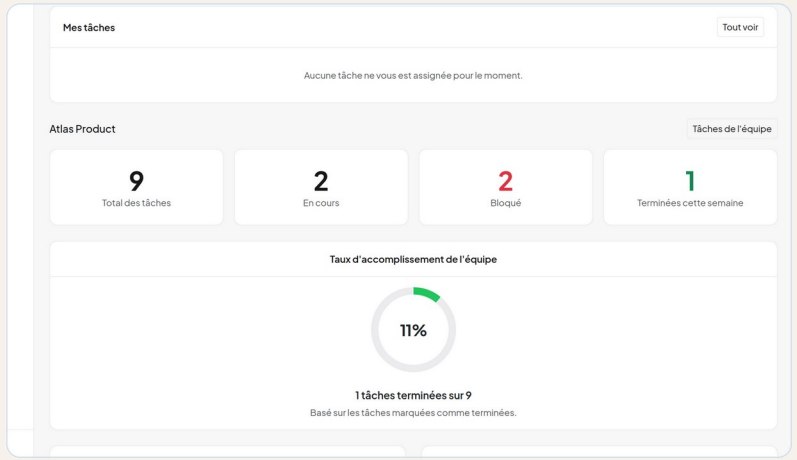
Résultat

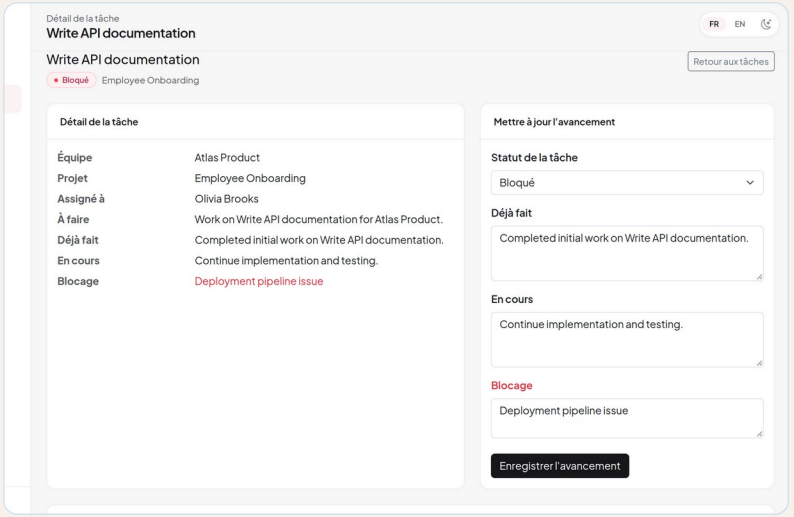
La réponse est 401. Avec un jeton valide, le membre lit son tableau.

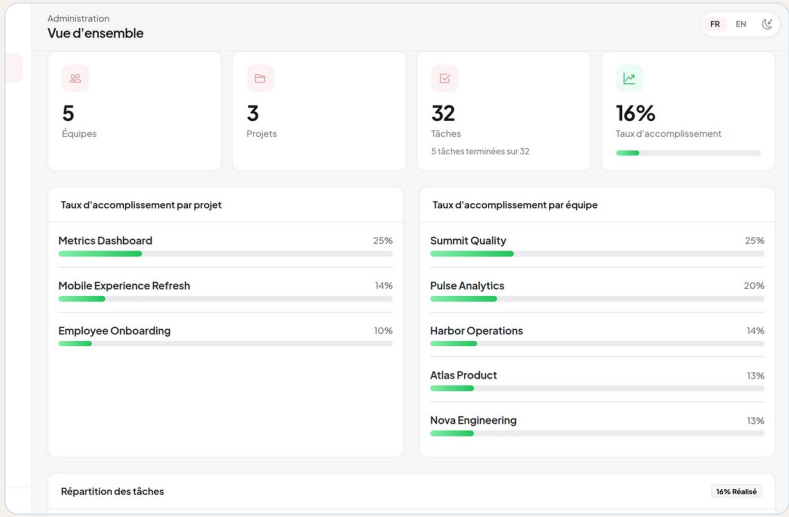
JsonEndpointTest.php

```php
$this->getJson(route('dashboard'))
    ->assertUnauthorized();
```

Démonstration







Chef d'équipe

Point d'avancement

Administrateur

En direct, environ 4 minutes.

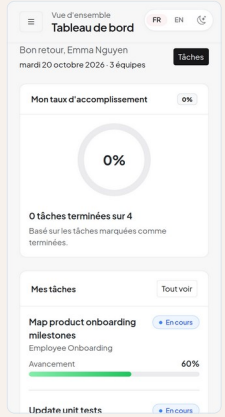
Déploiement

Serveur	PHP, racine sur public/
Base	MySQL ou MariaDB, migrations
Production	Pas de comptes de démonstration
Mobile	Même serveur, jeton Sanctum, HTTPS
Essais	lamraoui.alwaysdata.net

Bilan

Réponse

Suivi quotidien, sur la tâche, web et mobile,
avec des rôles distincts. Pas d'inscription publique.



Technique

Besoins, diagrammes, Scrum
Laravel, Flutter, PHPUnit

Professionnel

Du besoin jusqu'aux tests
Front-end et parrainage à côté

Pas livré notification, brief du jour, export, expiration des jetons

Réponse à la question

Merci pour votre attention.
Je suis à votre disposition pour vos questions.

Chiraz Lamraoui · Briefly · 27 octobre 2026